

Functions and Recursion in C++

[DEF] Definition to Remember

Functions in C++ provide code modularity, parameter passing (by value, reference, or pointer), function overloading, default arguments, and recursive call stack execution.

1. Concept Overview & Fundamentals

Functions and Recursion in C++ provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

* Ze

2. Technical Architecture & Memory Mechanics

C++ Runtime Execution Architecture



Modern C++ Feature Comparison

Modern C++ Paradigm	Legacy Equivalent	Engineering Advantage	Performance Impact
<code>**std::cin` / `std::cout`**</code>	<code>`scanf` / `printf`</code>	Type-safe, extensible for custom class	Buffered I/O (optimizable via <code>`ios::sy</code>
<code>**References (`Type&`)**</code>	Pointers (<code>`Type*`</code>)	Non-nullable, clean syntax without der	Direct alias; zero runtime overhead
<code>**std::vector<T>`**</code>	Raw Dynamic Array (<code>`malloc`</code>)	Automatic resizing, bounds safety, mem	Amortized <code>**O(1)**</code> push_back, contiguo
<code>**`nullptr`**</code>	<code>`NULL` / `0`</code>	Strongly typed pointer literal prevent	Zero overhead

3. Core Syntax & Implementation Patterns

Modern C++ Template Structure

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

// Fast I/O and standard namespace configuration
int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    std::cout << "Executing C++ Module: Function and Recursion" << "\n";

    return 0;
}
```

4. Real-World Applications & System Engineering

- * Quantitative Finance & HFT: High-frequency trading platforms use modern C++ for microsecond-level execution.

* Ga

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: add function

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Wri
 */

/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
 */

#include

// Use
```

```
using  
  
int ac  
    //  
    re  
}  
  
    //  
int ma  
    in  
    //  
    co  
    //  
    ci  
  
    in  
    //  
    co  
  
    //  
    re  
}
```

Problem 2: fibonacci recursion

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * CON  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 *  
 * 4.  
 * *  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
  
// Use  
using  
  
int fi  
    //
```

```
    if
    //
    if
    //
    re
}

//
int ma
in
//
co
//
ci

//
co
//
fo

}
//
co

//
re
}
```

Problem 3: get average function

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cal
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
 */

#include
```

```
// Use
using

double
//
re
}

//
int ma
in
//
co
//
ci

do
//
co

//
re
}
```

Problem 4: greet function

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Wri
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 * ==
 */

#include

// Use
using

void g
```

```
//
cc
}

//
int ma
gr
//
re
}
```

Problem 5: increment function

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Dem
 * inside
 */

/*
 * ===
 * COM
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ===
 */

#include

// Use
using

void i
n+
//
cc
}

void i
n+
//
cc
}

//
```

```
int ma
    in

    //
    co

    in
    //
    co

    in
    //
    co

    //
    re

}
```

Problem 6: max function

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
 */

#include

// Use
using

float
//
re
}

//
int ma
```

```
fl
//
co
//
ci

//
co

//
re
}
```

Problem 7: palindrome recursion

Problem Description & Objective:

Implem

C++ Code Implementation:

```
/*
 * Que
 * Cre
 */

/*
 * ===
 * COM
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ===
 */

#include

// Use
using

int re
//
if

//
re
}

bool i
//
if
```

```
//
re
}

//
int ma
in
//
co
//
ci

//
if

//
el

//
re
}
```

Problem 8: print date function

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cal
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
 */

#include
#include

// Use
using
```

```
void p
    ti
    ch
    //
    co
}

//
int ma
pr
//
re
}
```

Problem 9: square function

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Def
 */

/*
 * ===
 * COM
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ===
 */

#include

// Use
using

int so
//
re
}

//
int ma
in
//
co
```

```
//  
ci  
  
//  
co  
  
//  
re  
}
```

6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

1. Call-by-reference (type& ref) allows direct modification of caller variables without pointer syntax.
2. Function overloading enables multiple functions with the same name but distinct signatures.
3. Inline functions (inline keyword) suggest compiler copy-pasting function bodies to eliminate call overhead.
4. Recursion Stack Overflow: Occurs when base case is missing or recursion depth exceeds stack limit.
5. Divide and conquer algorithms (Merge Sort, Binary Search) rely heavily on recursive tree partitioning.

[RECALL] Quick Recall

Signature Overload -> Reference Passing -> Base Case Guard -> Recursive Call -> Stack Pop